

JAXtari: Create Novel Neuro-symbolic Methods for Solving Maze-based JAXtari Environments

Group ID: 13

Arshia Moradibirgani, Esther Song, Ameer Abou Saab

1 Introduction

Atari games are notoriously hard to solve, especially by Deep RL agents, because they lack generalization across different games[2]. However, Atari maze-based games can be split into two sections: maze navigation and enemy avoidance.

The *mazes* side itself is static and fully observable, and can be solved with conventional maze-navigation techniques. The *enemy behavior* requires picking up on patterns that are game-specific. This combination of features makes maze games the perfect environment for a neuro-symbolic split approach.

1.1 Problem and Research Question

We wanted to see the feasibility of developing a method that separates the problem into path-finding and danger avoidance, and how effective it would be when transferred to other games. The main research questions we faced were:

1. What type of method can solve maze-based environments under a neuro-symbolic reinforcement learning model?
2. Can a model's trained decision network based only on one type of environment be applied to a different environment with unique features without retraining?
3. Which parts of the agent transfer to other games, and which have to be rebuilt per game?

1.2 Approach

We chose an approach inspired by potential-field navigation. This meant that the problem would be separated into two parts: an *attraction* field that showed the agent where it needed to go in the maze, and a *repulsion* field that would tell the agent which parts of the maze it should avoid. Attraction would be calculated based on a goal mask, and repulsion would be created using a NN based on enemy position and other game-specific features. The navigation planner acts as the final decision-maker, using the combination of the two fields to shape the cost of traversing the maze.

The agent was implemented in a way that it would be compilable with JAX, and we used JAXAtari environments for testing and training. We worked to check whether solving mazes generalizes across environments. Ms.Pac-Man was chosen as the game where the agent would be trained. Then the agent would be tested on Pac-Man because of how similar the two games are, and we anticipated it would produce similar but somewhat diminished results. Then, it would be tested on Alien, since the layout of the game is similar to the two previous games, but Alien adds

new mechanics and enemy behavior. We expected the pathfinding to perform well, but we were not sure about the danger avoidance. Finally, the agent would be tested on the Bankheist game, which features completely different maze layouts and mechanics that are not similar to Ms. Pac-Man. We thought the navigation part of the agent would still perform well, and the agent might produce good results. But because of the different mechanics in the games, we didn’t expect the results to be the best.

2 Related Work

We needed ideas to solve two problems: getting symbolic information about the game state, and navigating the maze.

Through research on related material for an existing pipeline that could extract pellet and ghost locations directly from raw frames rather than building our own extraction logic, we found **SCoBots** [3], which, via the OCArari framework, extracts object-centric state through RAM parsing rather than vision. It was a much closer match to what our feature schema needs; this emphasized building our own lighter-weight extraction directly from JAXAtari’s own state and observation objects.

As for navigation, we needed to be able to separate our danger field from navigation; that is where **Potential Field Navigation**[6] comes in. This algorithm is used for robot navigation in which the system creates an artificial potential field where the goal acts as an attractor (low potential), and obstacles act as repellers (high potential), causing the robot to follow the negative gradient of the field toward the target while avoiding collisions.

The total potential $U(q)$ at any point q is the sum of an attractive potential U_{att} and repulsive potentials U_{rep} from all detected obstacles:

$$U(q) = U_{\text{att}}(q) + \sum_i U_{\text{rep},i}(q) \quad (1)$$

The resulting force $F(q)$ guiding the robot is the negative gradient of this total potential:

$$F(q) = -\nabla U(q) \quad (2)$$

However, working with gradients would add more difficulty to the project compared to more traditional maze-solving algorithms. **Dijkstra’s** algorithm [4] finds exact shortest paths on a graph with non-negative costs by expanding the cheapest unexplored node first, and **A*** [5] speeds this up with a heuristic that never overestimates the remaining cost.

We first planned to use A*, but both algorithms depend on priority queues and data-dependent loops, which JAX cannot compile. We therefore used value iteration [1, 7]: starting from $V = 0$ at the goals, a fixed number of parallel Bellman updates:

$$V(s) \leftarrow \min_{s'} [c(s') + V(s')] \quad (3)$$

will yield the same distances Dijkstra would compute, but with fixed-shape array operations that run inside `jit`. It also gives the cost to the nearest of many goals from every cell in one pass, and lets us add the learned danger term to the step cost c without changing the algorithm. Our model-based danger estimate used breadth-first search (Dijkstra with unit costs) per ghost.

3 Method

3.1 Implementation

The agent is implemented in JAX on top of JAXAtari. Every decision is built from two fields defined over the maze grid: *distance field* V produced by the planner, and a learned *danger field* produced by **DangerNet**(3.3).

Each game has its own encoder, which extracts the maze once per episode and, at every step, turns the current observation into a feature grid. The features are given to **DangerNet**, which outputs a danger field with the same shape as the maze. The danger field $D(s) \in [0, D_{\max}]$ is turned into a per-cell cost. Squaring makes it negligible far from enemies and steep close to them:

$$score(s) = V(s) + \lambda \cdot danger_{cost}(s) \quad (4)$$

$$a^* = \arg \min_{s' \in \mathcal{N}(s)} (score(s')) \quad (5)$$

where λ sets how strongly the learned danger is weighted against the distance to the goal.

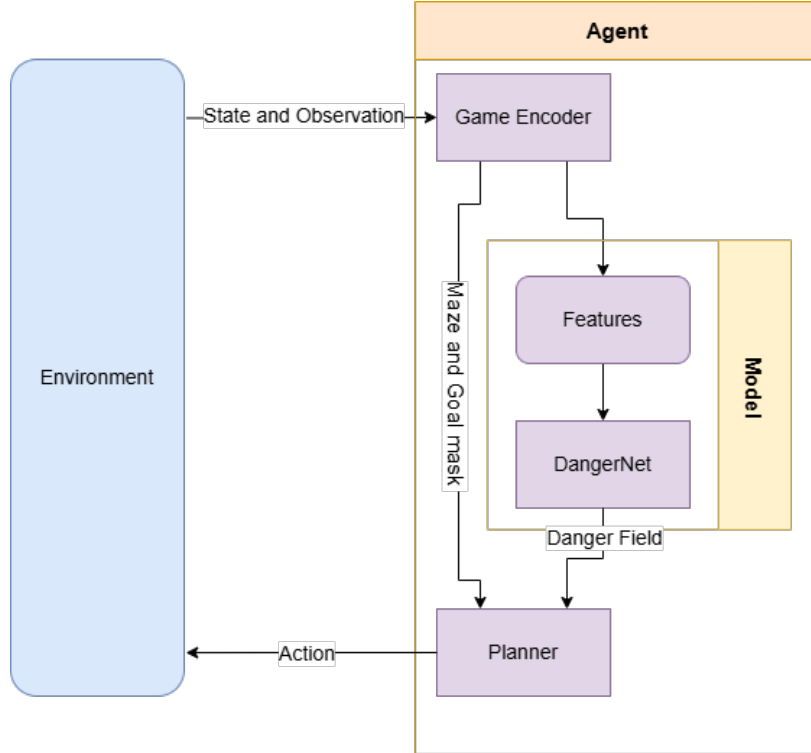


Figure 1: Agent Flow Chart

3.2 Encoders

Each encoder implements the same **GameEncoder** interface (`maze`, `walkable`, goal cells coordinates, with a set of game-specific functions `snap`, `goal`, `features`, `enemy_pos`, `player_pos`, `score`, `enemy_active`, and `valid`), so everything downstream of it is game-agnostic.

To ensure that the agent would work, three functions were needed from the Encoder: *Maze Extraction*, *Goal Selection*, and *Feature Extraction*

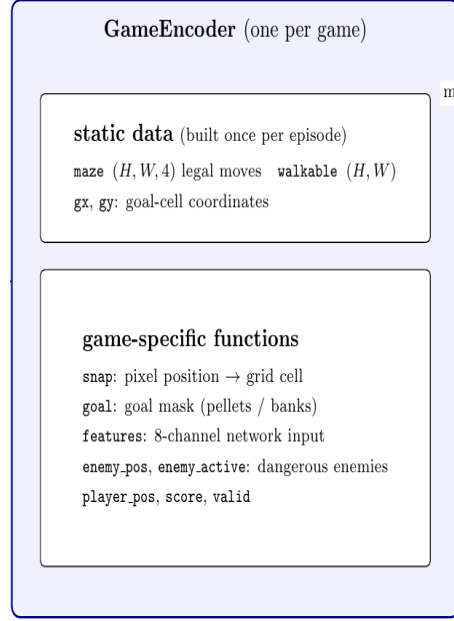


Figure 2: The *GameEncoder* interface. Each game supplies its own encoder which connects the planner, *DangerNet* and the training/evaluation code.

- **Maze extraction:** The encoder produces a boolean **walkable** grid and a legal-move array of shape $(H, W, 4)$. For games like Ms. Pac-Man, this comes from the game’s precomputed `DOF_MAZES`; for others, like Bankheist and Alien, it is computed from the game’s collision map with a sliding window that checks whether the player’s full collision footprint fits at each position.
- **Goals:** A handwritten goal mask is used for each game to indicate to the agent which maze cells it should move towards. For example, the remaining pellets in Ms Pacman, or the active banks in Bankheist.
- The **Features()** function assembles the 8-channel features that are then given to the *DangerNet* for calculating danger. Note that unreachable cells are marked with -1 :
 - Channels 0-3: legal movements (same as **Walkable**)
 - Channel 4: the agent’s distance to every cell
 - Channel 5: the distance from the nearest dangerous enemy to every cell
 - Channel 6-7: the nearest enemy’s movement direction and speed (v_x, v_y)

Because this schema is the same for every game, the same network weights can be loaded for any encoder.

3.3 DangerNet

DangerNet is a small fully convolutional net. It receives the encoder’s per-cell features and uses every channel except channel 4 (the agent’s distance to every cell) to output one danger value per maze cell, in $[0, D_{\max}]$. Because it contains only convolutions, the same weights run on mazes of any size (e.g. 40×44 for Ms. Pac-Man and 40×52 for BankHeist), and the general structures such as corridors and corners can be learned by the net.

The network has two 3×3 convolutional layers with 32 channels and ReLU, followed by a 1×1 convolution and a sigmoid scaled by $D_{\max}$. The output is gated: if no dangerous enemy is reachable, the field is set to zero. It is trained with REINFORCE on Ms. Pac-Man using a shaped reward (Section 4).

3.4 Planner

The planner acts as the final decision maker. It uses the maze **walkable** and the **goal** alongside the **DangerNet** output to select the best possible action by using greedy action selection.

Given the danger field $D(s) \in [0, D_{\max}]$, danger is turned into a per-cell cost. Squaring makes it negligible far from enemies and steep close to them:

$$c(s) = \frac{D(s)^2}{D_{\max}} \quad (6)$$

Then goals that are inside danger areas are dropped. If by removing the goals, no goal would remain, then all goals are kept:

$$G' = \{g \in G : c(g) < \tau\} \quad (7)$$

Then Value iteration runs for a fixed number of sweeps with the danger as an additional traversal cost and the new goals.

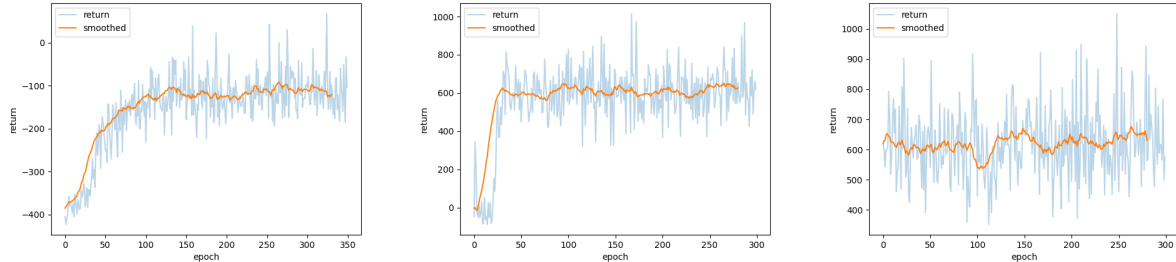
$$V(s) = \begin{cases} 0 & s \in G' \\ \min_{s' \in \mathcal{N}(s)} [1 + \lambda \cdot c(s') + V(s')] & \text{otherwise} \end{cases} \quad (8)$$

And finally, the agent moves to the neighbour with the lowest value in the greedy step:

$$a^* = \arg \min_{s' \in \mathcal{N}(s)} V(s'). \quad (9)$$

We use $\lambda = 8$, $D_{\max} = 10$ and $\tau = 8$. With these values, a goal is dropped once its danger exceeds about $\sqrt{10} \approx 3.2$.

3.5 Training



(a) Training model V:2 - early versions of reward shape (b) Training model V:4 addition of aux-loss. (c) Training model V:6 - final model created

Figure 3: Training curves representing the jump between model versions

The Training was done on Ms. Pac-Man maze 0. Mazes 1-3 were used for the generalization test. For **Policy**, we used a stochastic decision rule for training. We kept it simple by only checking

for danger at the four immediate neighbors rather than routing it through the planner (which we treat as a constant with no gradients):

$$\pi(a \mid s) = \text{softmax}_{\text{legal } a} \left(- [V_{\text{nav}}(n_a) + \lambda \cdot c(n_a)] \right) \quad (10)$$

where

$$n_a = f(s, a) \quad (11)$$

We used JAXAtari’s own reward with some modifications. We heavily penalized deaths ($P_{\text{death}} = 50$) and added a shaped penalty ($w = 0.05$) for getting within $R = 10$ steps of a ghost.

$$r'_t = r_t - P_{\text{death}} \cdot \mathbb{I}[\text{died}_t] - w \cdot \max(R - d_t, 0) \quad (12)$$

As for the loss, we combined standard REINFORCE loss with an auxiliary MSE loss (scaled by $\beta = 0.05$). Without the auxiliary loss, the network just memorized the maze geometry (like learning that corners are inherently dangerous) instead of actually tracking the moving ghosts. This auxiliary term forced the model to anchor its danger map to the enemy positions rather than maze geometry.

Training was done on Ms. Pac-Man (maze 0) for 300 epochs. A random warm-up (between 0 and 60 steps) was added to mix up the starting states to add variability, and 8 parallel environments (800 steps each) using Adam (lr 3×10^{-4} , clip 1.0) were run; the checkpoint with the best return was kept. The whole setup — batch size, network, and episode length — was specifically squeezed down to fit on a laptop RTX 3060 with 6 GB of VRAM. For speed, the entire rollout is JIT-compiled and `vmap`’d across environments.

4 Experiments and Results

4.1 Navigation and Handcrafted Danger

To show that our idea works, we decided to do two experiments: first, to test the navigation component, we ran the agent with a zero danger field. This meant that the calculated $V(s)$ would act as only navigation, with no danger cost. Then, we created a handcrafted danger field, which would create a fixed danger radius around the ghosts, with the center being the highest point of danger and fading out as it moved outwards.

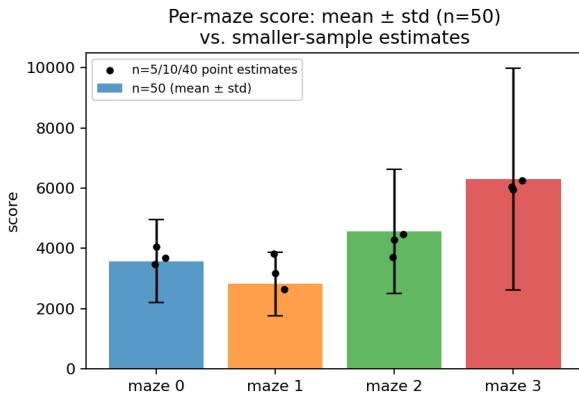
The results in the Table¹ show that, by adding the handcrafted danger, the score nearly doubles, meaning that the two field approach is viable.

Maze	None		Handcrafted		Learned	
	Score	Surv.	Score	Surv.	Score	Surv.
0	1415 $\pm$ 530	1579	3549 $\pm$ 1463	3469	3570 $\pm$ 1375	3745
1	1817 $\pm$ 861	1608	2521 $\pm$ 645	2571	2821 $\pm$ 1056	3046
2	1331 $\pm$ 712	1387	3189 $\pm$ 2033	2895	4561 $\pm$ 2059	3840
3	3179 $\pm$ 2702	1659	5428 $\pm$ 3766	3141	6304 $\pm$ 3686	3578
All	1935 $\pm$ 1660	1558	3672 $\pm$ 2526	3019	4314 $\pm$ 2628	3552

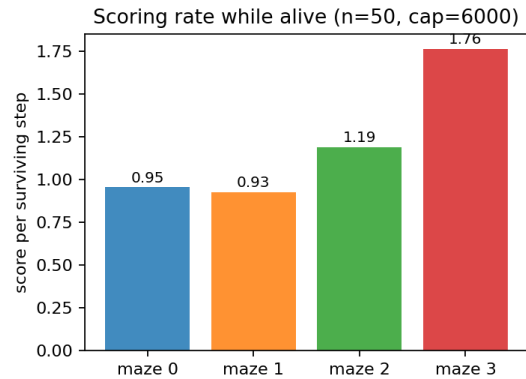
Table 1: Ms. Pac-Man mean score ($\pm$ std. dev.) and mean steps survived, 50 seeds per maze, cap 6000 steps. Learned: network trained on maze 0 only. The last row pools all 200 episodes, so its deviation includes differences between mazes.

4.2 Learned Danger on Ms. Pac-man

After our initial experiment, several variations of the agent were created and tested. The results below use the newest version of the models (v6) resulting checkpoint, trained on maze 0, evaluated with the learned danger source across all four maze layouts:



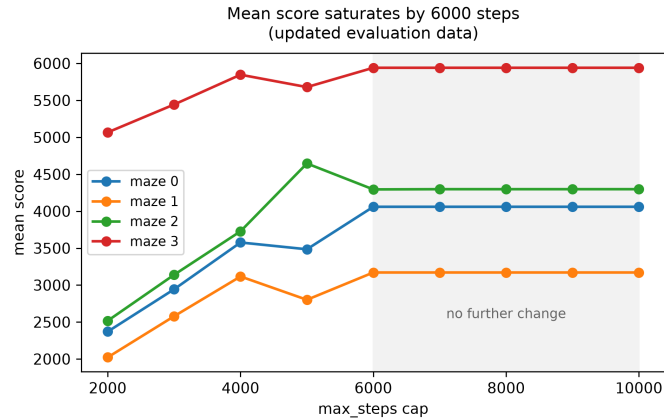
(a) Per-maze spread



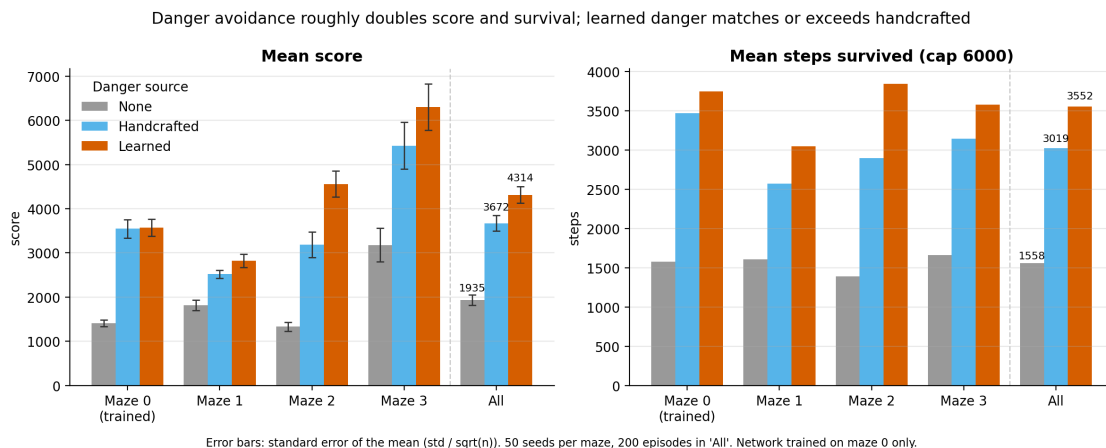
(b) score rate

Similar to the no danger and handcrafted danger, there is high variance between the scores, especially in maze 3. The learned danger has a 17.5% better overall score and 17.6% better survivability. However, this lead is uneven between different mazes. The most gains seem to

be in Maze 2, while Maze 0 and 1 have marginal improvements. The net matches handcrafted on the maze it was trained on and pulls ahead on unseen ones.



(a) Max-step saturation



(b) Ms. Pac-Man results with three danger sources. Left: mean score. Right: mean steps survived (cap 6000). Error bars show the standard error of the mean; 50 seeds per maze. The network was trained on maze 0 only.

We also decided to test the episode cap to see if it resulted in any changes. We saw that after `max_step=6000`, the means stop changing, and the maximum survival limit is below 6000 steps.

4.3 Problems with the Agent

Reward Hacking When the **DangerNet** was trained only on the environment’s raw reward, it did not produce avoidance behavior. This was because the game’s reward came only from eating the pellets, and danger avoidance behavior was not rewarded. Also, dying is a sparse event (only happens three times before an episode ends); this caused the agent to learn that the best strategy was to just eat pellets even if it meant dying to ghosts.

We then decided to shape the reward so that it matched our desired behavior. We decided to add penalties for being near ghosts and dying. This fixed the ignoring-ghosts problem, but introduced a new one where the agent learned to retreat to a region of the maze the ghosts rarely reached and oscillate back and forth there for the rest of the episode, surviving indefinitely while leaving most pellets uneaten (score around 750). Figure 6 shows the exact position that the agent would oscillate on to avoid the ghosts indefinitely.

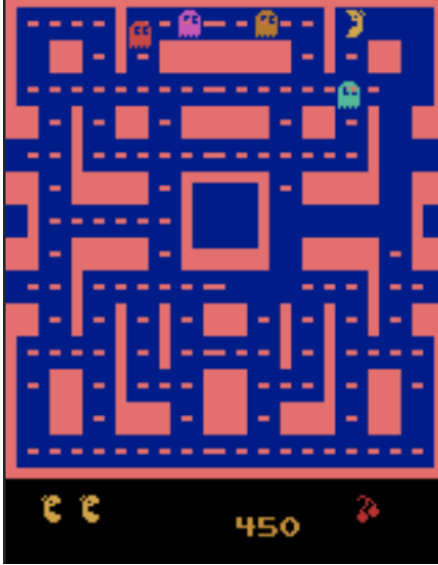
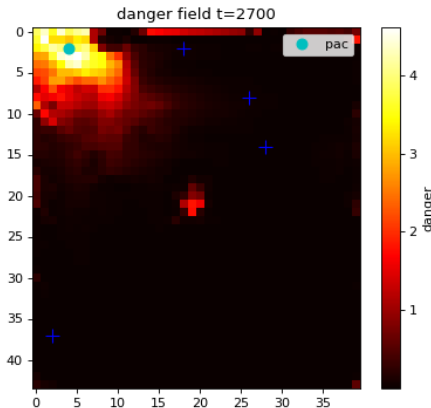


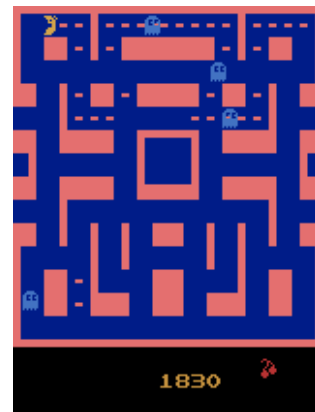
Figure 6: Every cell visited during a full 3000-step episode under the proximity-penalty reward. The agent would oscillate in this position indefinitely, and the ghosts would circle the top middle of the maze and would not kill the agent.

Pixel Perfect Interactions In the Atari games, a collection of pixels is turned into one cell to create a maze layout. For example, in Ms. Pac-Man, every four pixels are grouped into one cell. This caused an issue with eating the pellets. When the agent would enter a cell with a pellet in it, it would think that it had eaten the pellet and would move on. Most of the time, this meant that it would move in the same direction and would move over all the pixels in the cell. However, sometimes the agent would try to return to the previous cell after entering only one pixel into the new cell. Because eating a pellet is a pixel-specific interaction, this caused the agent to think that it had eaten the pellet when in reality it had not. This then caused the agent to go back and forth between the two cells (the one with the not-eaten pellet and the next cell). To fix this, a coasting rule was added so the agent keeps its direction while standing on a goal cell until it has crossed it. However, this showed that certain pixel-specific interactions could cause problems in other games as well.

Degenerate Danger Signal ("I-am-the-danger") Another major bug we hit was nicknamed "I-am-the-danger". This happened whenever the danger term contributed little or nothing, like when pacman ate a power pellet and the ghosts therefore lost their danger value. The only remaining dense training signal was the navigation distance field itself, and the network picked up on this degenerate cue rather than on genuine danger.



(a) Bad danger heatmap



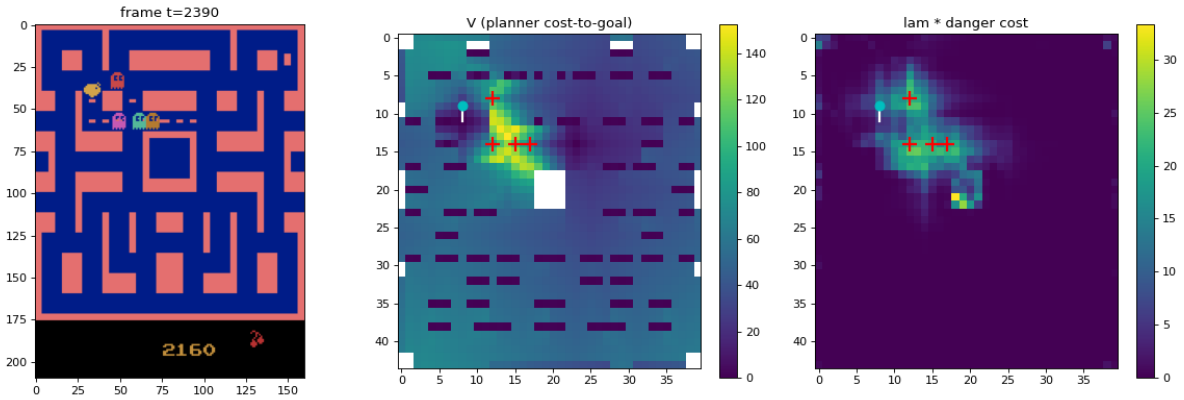
(b) "I-am-the-danger" freeze frame

Figure 7: Showcases how the danger score would invert and represent any adjacent cell to Ms. Pac-Man as dangerous; with a freeze frame to reference the heat maps' representation of the bad danger signal,

There were two proposed fixes: Normalizing the input channels, or dropping some of the encoder feature channels. The agent performed worse after normalizing, so the approach was dropped. But by adding a **gate** before the output, we made sure that when there is no danger present, the net doesn't output any residual danger. This somewhat solved the *I-am-the-danger* problem.

However, the gate didn't fully fix the problem. We realized that channel 4 (distance to agent) in the features was dominating the inputs, so we decided to drop channel 4 from the net's (3.3) input. This resulted in better performance (v5 and v6 models).

Danger is Reactive Another problem, or rather feature of the agent is that it has learned danger as a reaction to proximity to enemies. This behavior was created as a result of our reward shaping. Because the agent cannot predict danger, it can fall into traps where moving in any direction will eventually cause the agent to die (i.e. it is cornered by ghosts). Although this problem is bad, making the agent predict enemy movement might require it to learn game-specific features, which could hurt transferability to other games. Therefore, we decided to allow this limitation in the agent in favor of it being more transferable across different games.



(a) Example of dying to a pincer maneuver

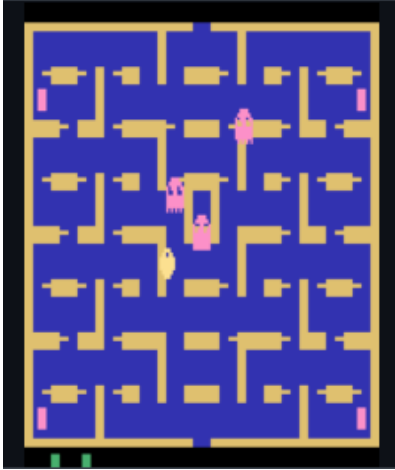
4.4 Transfer to Different Games

Ms. Pac-Man was the biggest focus of the project, with the majority of training and testing done on it. We then tested the agent on 3 different games: Pac-Man, Alien, and Bank Heist. Almost all environments transitioned poorly due to core concept difference, except for Pac-Man, as it and Ms. Pac-man share the core game-play loop. The poor initial transition was caused by a design limitation in our Encoders, which meant the agent couldn't be fully realized or be tested on an environment. Detailed results for each game can be seen in each subsection:

4.4.1 Pac-Man

Writing the Encoder was simple since Pac-Man uses a DOF maze pellet-array similar to Ms. Pac-Man. Since the two games have a similar layout and game mechanics, the agent managed to perform reasonably well on both Pac-Man mazes. The results in 2 show how the agent performs better with learned danger. JAXAtari's Pac-Man implementation defines two maze layouts; levels 1-2 use Maze 0, levels 3-4 use Maze 1. We therefore evaluated on two mazes. We usually rendered some runs to make sure that the agent was performing correctly. However, Pac-Man had rendering problems. The JAXAtari `render()` function was displaying the pellets and maze layout incorrectly.

That is why we are somewhat unsure about the Pac-Man results, even though the scores show good transfer.



Pellets, Pac-Man, and the ghosts do not align with the rendered maze walls

Maze	None		Handcrafted		Learned	
	Score	Surv.	Score	Surv.	Score	Surv.
0	12 ± 0	768	75.4 ± 38.9	2609	79 ± 55	2092
1	12 ± 0	768	86.6 ± 40.2	2959	104.5 ± 88	2274
2	12 ± 0	768	89.6 ± 44.1	2943	99 ± 83.1	2323
3	12 ± 0	768	88.2 ± 46.4	2916	91.2 ± 54.1	2222
All	12 ± 0	768	84.9 ± 42.9	2857	93.4 ± 72.4	2228

Table 2: Pacman: mean score ($\pm$ std. dev.) and mean steps survived, 10 seeds per maze, cap 3000 steps. JAXAtari’s Pacman defines two maze layouts. The learned network was trained on Ms. Pac-Man only and applied without retraining. Without danger, all episodes are identical (std. dev. 0). The last row pools the 40 episodes.

4.4.2 Alien

The game Alien has a similar layout and mechanics to Ms. Pac-Man. However, there is also an additional shooting mechanic in this game. After eating the special pellet (similar to the power pellet), the agent can now use the shoot action. This is not compatible with our design since our planner outputs only directional movement and doesn’t know what shooting is.

Danger source	Score	Survival	Score range
None	30.0 ± 0.0	1308	30 - 30
Handcrafted	840.0 ± 228.6	1819	560 - 1030
Learned	805.1 ± 269.4	1610	650 - 1561

Table 3: Alien (single maze): mean score ($\pm$ std. dev.), mean steps survived and score range, 5 seeds, cap 3000 steps. The learned network was trained on Ms. Pac-Man only.

The results in 3 show that the danger did transfer; the learned danger is producing better results than the handcrafted danger. Initially, the danger field produced by the Net was not to scale with the maze shape and size. We managed to solve this by rescaling the danger so that it performed better and managed to produce better results than the handcrafted danger. This means that the agent managed to transfer between the two games after correctly adjusting the Encoder.

4.4.3 Bankheist

Before testing cross-game weight transfer, we validated the symbolic half of the pipeline for Bank Heist in isolation, with the danger term set to zero. The encoder resolves a 40×52 grid with 374

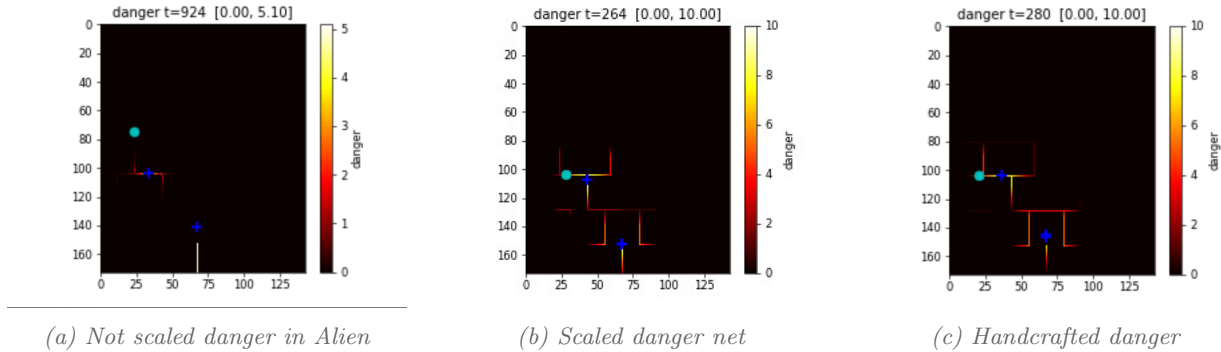


Figure 9: Showcases the difference between the danger fields produced by Alien encoder.

of 2080 cells walkable (18%).

We then replaced the zero danger term with DangerNet, loaded from the v4 checkpoint trained only on Ms. Pac-Man, with no retraining or fine-tuning on Bank Heist. Getting to this point was itself non-trivial: an earlier version of the Bank Heist encoder only produced 3 feature channels against Ms. Pac-Man’s 8, and even manually padding 3 channels out to 8 for a smoke test was not enough to get a working checkpoint load. The encoder used here was rebuilt to match the 8-channel schema natively.

At the fusion weight used during Ms. Pac-Man’s training ($\lambda = 8$), the agent became almost completely immobile: over a 2081-step episode, the total set of grid cells it ever occupied was confined to essentially its spawn point, and no money was earned (Figure 10).

However, even after the encoder changes and fixes, the agent would still only score 60. We learned that this is because each Bankheist maze spawns only three banks (which are goals), and after eating them, the agent would stay idle because it didn’t know where it needed to go. To fully test the game, the agent would need to be able to move to the other mazes (cities) and collect other banks as well. So while the agent managed to beat the first maze easily, it didn’t know what to do afterwards.

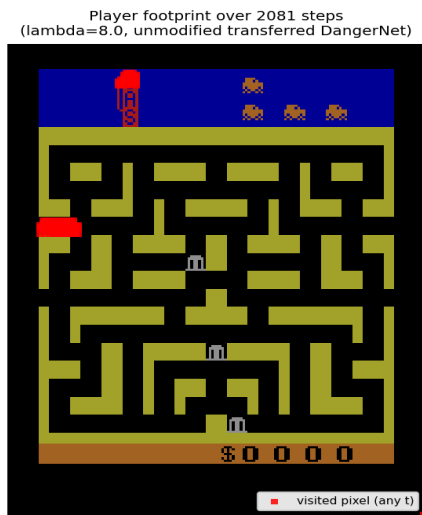
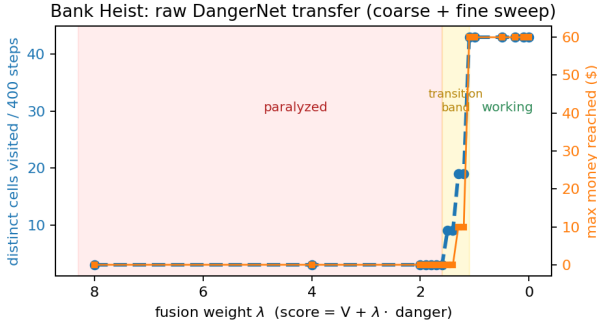


Figure 10: Every pixel that changed at any point during a 2081-step episode, using the Ms. Pac-Man DangerNet checkpoint at $\lambda = 8$. The visited area is confined to a tiny region near the spawn point (in red). the test concluded with no money earned (or banks reached).

Lambda Test. To characterize this, we swept λ , first coarsely and then finely, measuring distinct cells visited and maximum money reached over a fixed 400-step budget (Figure 11). Coarse sweep suggested a hard binary switch: fully working at $\lambda \leq 1$, fully paralyzed at $\lambda \geq 2$, nothing in between. A finer scan between those two points corrected that impression: there is a real, if narrow, transition band between $\lambda \approx 1.1$ and $\lambda \approx 1.6$, where the agent covers a partial area and earns partial money before collapsing fully. We also tested two candidate explanations for the paralysis at $\lambda = 8$ and ruled both out: increasing the policy’s directional momentum by three orders of magnitude did not restore movement, arguing against simple step-to-step output jitter as the cause; and masking channels 6–7 (the enemy movement-direction channels, flagged in advance as the weakest link in the schema) also did not restore movement, showing that channels 0–5 alone are already sufficient to dominate and break the fused score at that weight.



Fusion weight λ	Cells visited	Max money
1.90 and above	3	\$0
1.60–1.80	3	\$0
1.40–1.50	9	\$0
1.20–1.30	19	\$10
1.10 and below	43	\$60

Figure 11: Combined coarse and fine sweep of the fusion weight λ with the raw, unmodified transferred DangerNet. There is a genuine transition band around $\lambda \approx 1.1$ – 1.6 , narrower and less smooth than we would want for practical tuning, but not the pure on/off switch the coarse sweep alone suggested.

We separately tested per-frame normalization of the danger field (z -scoring it before multiplying by λ) as a lightweight, no-retraining fix, which also produced working behavior below approximately the same $\lambda \approx 1.1$ threshold. Whether this reflects a genuinely informative signal that simply needs correct scaling, or a signal that is small enough at low λ to be practically inert either way, is what we are checking next, by directly comparing the network’s highest-danger cells against the true positions of active police cars rather than inferring transfer quality only from movement statistics.



Figure 12: tracing of the working Bankheist test run after lambda sweep experimenting

4.5 Problems with Transferability to different environments

4.5.1 Encoders and maze extraction

Encoders were used to extract features and mazes from the games' environment as explained in the previous segments 3.2. Making sure that the correct **Walkable** maze was generated was a huge problem. Additionally, we computed the maze extraction only once; therefore, the maze layouts created were static. Meaning that we couldn't handle dynamic maze changes which happen in Bankheist.

In addition to the maze extraction, another problem we faced in both Alien and Bankheist was generating the **Walkable** maze layout from the collision map. Additionally, since the agent was trained on Ms. Pac-Man, it had learned to perform on that pixel-to-cell scale. Meaning that we needed to find a way to re-scale the danger for each maze. This could be done either in the Encoder (similar to what we did in Alien) or by adjusting the λ values. We believe a combination of these two would provide the best result for each maze without the need for re-training.

12

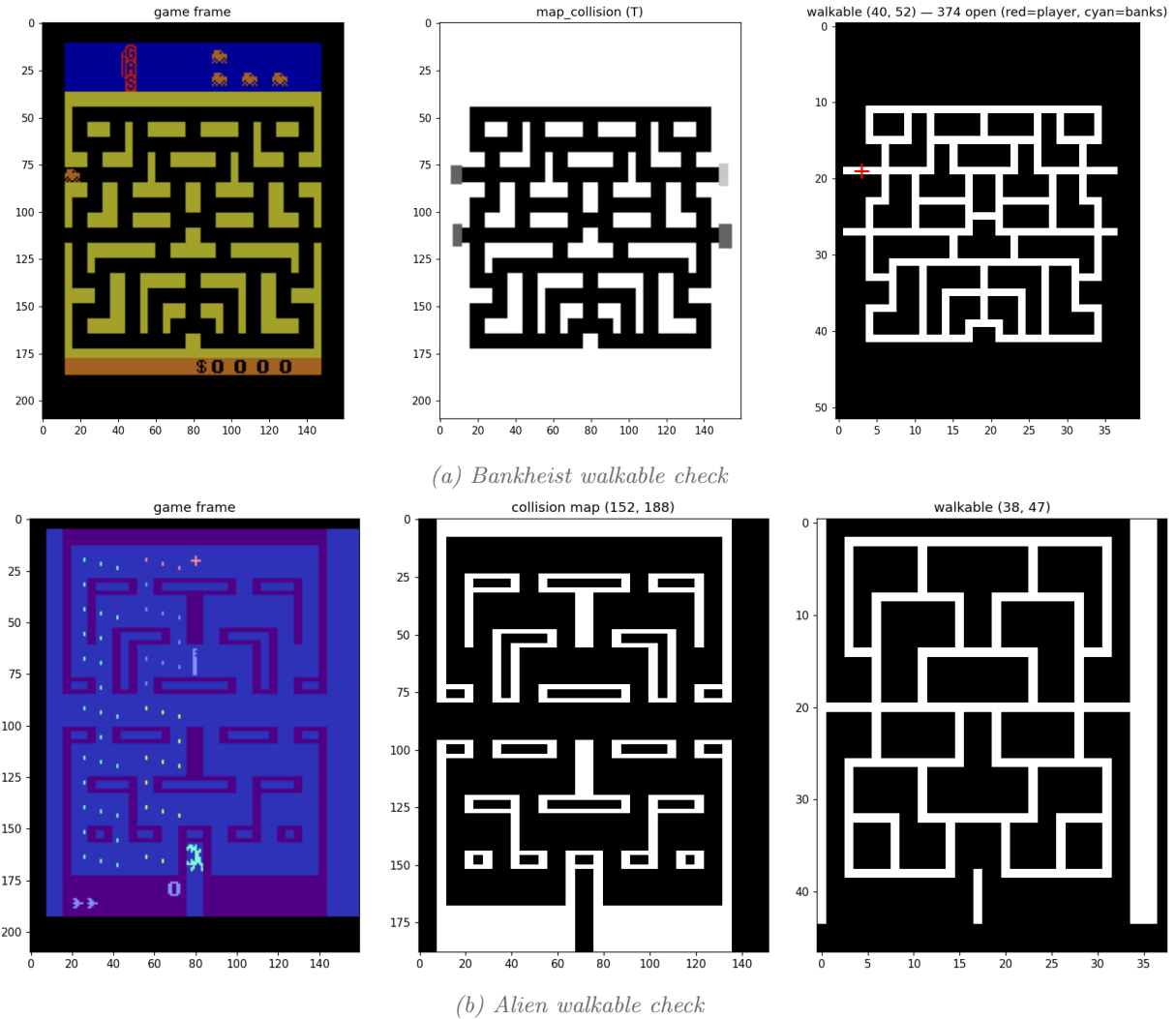


Figure 13: Examples of extracting **Walkable** from collision map on Alien and Bankheist

4.5.2 Goal selection and dynamic goals

Bankheist's unique game-play loop provided a new challenge. Similar to Ms. Pac-Man, its goals were in the form of banks placed particularly around the map, with a robber (or the player) trying to rob them without getting caught by the chasing police car(s). The problem was that Bankheist had multiple mazes, and only three Banks would generate in each maze. Therefore, additional conditions to the goal needed to be added so that when all the Banks were robbed, the agent would move to the next maze.

Refactoring to the new goals was simple and required little effort; however, the transition to the next scene was an arduous task that was not appropriately handled by the agent without major changes to the overall Encoder structure. This also required the Encoder to recompute the maze each time the agent entered a new maze, which was not compatible with the current implementation of our encoders.

Additionally, dynamic goal setting and selection applied to the other games as well. In the game **Alien**, the player can obtain a weapon to shoot the enemies to repel them. This is similar to when Ms. Pac-Man eats a power pellet. However, the shooting is a separate action from movement. Since our agent used the planner as the final decision maker, it can only do one of the four movement actions in Atari games (up, left, down, right). This means that games with unique interactions can not be handled by the agent.

5 Insights and Conclusion

The results show that the overhaul design of the agent works. We believe that the main reason is the symbolic nature of inputs that **DangerNet** receives and the fact that danger is decoupled from navigation. This shows that by abstracting features that are similar across different games (maze generation, navigation, avoiding danger), we can increase the chance of transferability.

However, more transferability comes at a cost. Certain game-specific behavior cannot be used across multiple games while maintaining transferability. Specific ghost behavior in Ms. Pac-Man and shooting enemies in Alien are good examples of this. While the agent might perform better if it knows about power pellets or about the shooting mechanism, these specifics are not good for abstraction.

Having λ as a value that can be adjusted for every maze helps with transferability as well. We believe that each maze (based on size and scale) can benefit from fine-tuning λ . It also allows for the **DangerNet** to produce danger in a specific range, which we can then adjust for each maze size using λ .

The main problem with our approach is that the Encoder itself needs to be created individually for each game to allow for the transfer of the navigation and the danger. Dynamic goals and maze generation can be handled easily by changing some aspects of the Encoder design, but because of time constraints, we decided not to tackle this problem.

For future work, we recommend an approach where the current agent acts as a bootstrap for creating more specialized agents for each environment. The danger can be used for enemy avoidance, but a NN on top of the planner can be trained for each game to both help with dynamic goal selection and dealing with extra or game-specific actions and mechanisms.

Our major implementation handicap was JAX itself. While it provided us with faster testing and training using GPU, it forced us into a specific design pattern and choices. For example, not being able to use A* and having to use value iteration instead. JAX also has problems with WSL and some GPUs that are minor problems but caused delays in testing and development of the code.

Contributions

- Shared interface, planner, and navigation design decision: **Arshia**
- The Game Encoder interface and code consolidation: **Arshia**
- Model-based (BFS-per-ghost) danger estimation: **Esther, Ameer**
- **DangerNet** (MF approach) and REINFORCE training on Ms. Pac-Man, including the reward-design iteration: **Arshia**
- Pac-Man encoder heads, and cross-game evaluation: **Esther**
- Bankheist encoder heads, and cross-game evaluation: **Ameer**
- Alien encoder heads, and cross-game evaluation: **Esther**
- Infrastructure and tools (batched updates, debug and heatmap visualization): **Arshia**
- Report: **Ameer, Arshia, Esther**

References

- [1] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [2] Quentin Delfosse, Jannis Blüml, Bjarne Gregori, and Kristian Kersting. HackAtari: Atari learning environments for robust and continual reinforcement learning, 2024.
- [3] Quentin Delfosse et al. Scobots (as referenced by the group; todo verify exact title/author list before submission), 2024. Cited by the group for its RAM-parsing (OCAAtari-based) route to object-centric state, as an alternative to vision-based object discovery.
- [4] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [5] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [6] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986.
- [7] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.